

01 | Readiness verdict

What this project is

ClaimLens is a human-in-the-loop document review prototype for an insurance claims reviewer. It compares a hospital bill, discharge summary and supporting reports, identifies discrepancies, and presents traceable source evidence. It does not decide whether a claim is fraudulent, authentic, approved, rejected or payable.

Honest assessment

Area	Assessment
Local demonstration	Prepared for a synthetic walkthrough. Automated interaction and business-rule checks pass.
Live AWS demonstration	PARTIAL. The stack and Amplify frontend are deployed in ap-south-1; Cognito login and a protected API read pass. Textract and Bedrock remain blocked by AWS account activation.
Browser appearance	VERIFIED for the explicit local mock flow at desktop/mobile widths and the production Cognito/empty-queue flow at desktop width. Signed-source rendering remains unverified.
Hackathon eligibility	NOT VERIFIED. The event name, judging rubric, deadline and submission rules have not been supplied.
Production medical use	NOT READY. Needs operational, privacy, security, extraction-accuracy and cloud acceptance work.

What you can confidently present

A working synthetic reviewer journey, source-backed arithmetic and cross-document checks, a consistent paper-charcoal-lime interface, explicit uncertainty, strict citation validation, AWS SAM infrastructure and a reproducible test suite. Demonstrate the distinction between review priority, extraction quality and coverage.

What not to claim

Do not claim completed live document processing, measured fraud-detection accuracy, insurance approval automation, clinical validity, regulatory compliance or proven OCR performance. The AWS foundation is deployed, but Textract/Bedrock execution is not yet available. No real medical records should be used for this hackathon prototype.

The assessment date and exact verification results appear in the footer and on the verification page. Passing local checks does not remove the cloud-readiness gate.

02 | Website and reviewer journey

Main screens

Screen	What it does
Landing page /	Split-image Aero hero, synthetic example, architecture-service strip and entry to the review workspace. Sample figures are not real customer metrics.
Review queue	Lists authorized packets, review priority, attention checks, coverage and document count. Search narrows the displayed packets.
Claim review	Shows five-category check outcomes, selected finding, citations, extraction quality and coverage. Reviewer actions remain separate from automated results.
Documents	Lists document type, version, pages, extraction state and quality. View evidence opens a cited source when available.
Source viewer	Renders original PDFs/images in production with a cited region overlay. Mock mode shows a labeled evidence sheet, not an original PDF.
Activity and report	Shows reviewer actions and correction annotations. Export produces a JSON review record; it is not a signed PDF claims report.

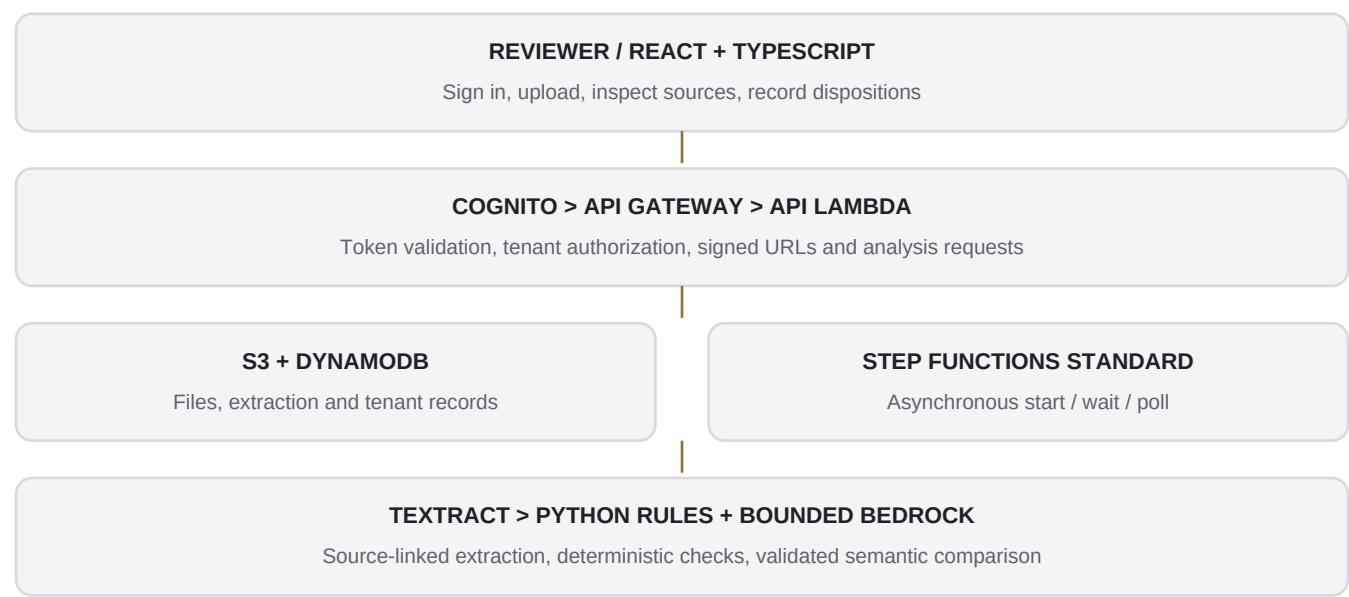
A normal review

1. Open a sample packet, or submit one bill plus optional discharge/supporting documents.
2. Inspect analysis status, review priority, extraction quality and check coverage independently.
3. Select a check and follow every cited document/page. Inspect confidence and block references.
4. Record Open, Acknowledged or Resolved. These dispositions never adjudicate a claim.
5. Add a correction if extraction is wrong. It preserves the original and does not rerun checks yet.
6. Export the latest JSON report and review missing evidence requests with a human.

Upload behavior

The interface allows PDF, PNG and JPEG, up to 15 MB per file, no more than 10 documents, and exactly one itemized bill. Each file has an explicit type. Mock uploads do not read, upload or OCR the file bytes: they create an extraction-unavailable result. Use built-in samples for the complete offline demonstration.

03 | Architecture at a glance



Responsibilities

Component	Responsibility
React + TypeScript + Vite	Reviewer interface, upload coordination, status polling, evidence inspection, corrections and report download. The static frontend host is not provisioned by this SAM template.
Cognito + API Gateway	Sign-in via authorization code with PKCE; gateway validates JWTs before Lambda derives tenant identity.
API Lambda	Tenant authorization, claims/documents, signed S3 URLs, analysis requests, review actions, corrections and report assembly.
S3 + DynamoDB	Private versioned source/raw/normalized files in S3; tenant-scoped application records in DynamoDB.
Step Functions Standard	Asynchronous Map/Wait/Poll/Analyze processing with explicit failure handling.
Textract + Bedrock	Textract extracts forms/tables/text; Bedrock makes bounded source-cited semantic comparisons. Python owns arithmetic and deterministic rules.

Why this split matters

Long OCR jobs do not hold the upload request open. Large source files stay outside the workflow state. Deterministic reconciliation remains reproducible even when AI is unavailable. The model has no write tools, browser access or direct database access.

04 | What happens after an upload

Request and processing sequence

1. The UI signs in with Cognito in production. It sends the current ID token to API Gateway.
2. POST /claims creates a deterministic claim identifier using a tenant-scoped idempotency key.
3. The upload-request endpoint registers each document and returns a short-lived presigned POST. S3 enforces content type, encryption and a 1-byte-to-15-MB range before accepting the browser upload.
4. POST /analyses checks ownership, verifies S3 size/content type and pins the uploaded VersionId. It returns HTTP 202 with an analysisId and initial status.
5. Step Functions runs up to three document branches at a time. StartExtraction calls Textract with the pinned S3 version and an idempotent client token.
6. The workflow waits five seconds, then polls Textract. Completed results are retrieved across all NextToken pages. Full raw and normalized extraction archives go to S3.
7. Analyze loads successful document extractions, runs deterministic checks, optionally calls Bedrock and validates its output. Partial or failed extraction cannot count as clean coverage.
8. DynamoDB stores the result and analysis snapshot. The UI polls the selected pending analysis every three seconds and displays its outcome.

Backend file map

File	Role
api.py	HTTP routing, tenant boundary, signed URLs, analysis/report view and reviewer updates.
workflow.py	Textract start/poll, raw archival, form/table adapters, semantic invocation and failure handler.
service.py	Claim/document/analysis operations, orchestration, finding assembly, snapshots and corrections.
repository.py	In-memory and DynamoDB repositories, conditional creation, tenant keys and request leases.
extraction.py + money.py	Provenance, field normalization, explicit date order and integer-paise conversion.
rules.py + semantic.py + models.py	Deterministic checks, bounded model contract and typed evidence/finding validation.

05 | Evidence and application records

Evidence is part of the data, not a decoration

Every normalized field refers to a versioned document, page, source block IDs, normalized bounding box, extraction confidence and excerpt. Evidence identifiers are derived from the document/version/page/blocks. A multi-block region covers the union of its source boxes. Missing geometry is rejected rather than invented.

Evidence member	Meaning
documentId / documentVersion	Identifies the logical source record. The physical S3 VersionId is pinned separately on that record.
page / blockIds	Gives the exact page and Textract blocks used for the value.
geometry	Left, top, width and height in normalized page coordinates. Nonfinite and out-of-page coordinates are invalid.
confidence	Textract confidence for the cited source. Not the probability of fraud or medical truth.
excerpt / evidenceld	Human-readable source text and stable citation key. Model references must resolve to known evidence.

Record model

DynamoDB uses `pk = TENANT#tenant` and typed sort keys. CLAIM, DOCUMENT, ANALYSIS, ANALYSIS_VERSION, REVIEW, REVIEW_EVENT, CORRECTION, INVOICE_FINGERPRINT and IDEMPOTENCY are distinct record kinds. Lookup authorization is derived from the validated token, not a request body tenant ID. Each record receives an `expiresAt` timestamp for DynamoDB TTL.

S3 stores original bytes, complete Textract responses and normalized extraction archives under `tenant/claim/document/version-oriented` prefixes. Bucket versioning is enabled. A new upload does not replace the pinned bytes used for an existing analysis. The configured retention period expires current and noncurrent objects; incomplete multipart uploads are aborted after one day.

Current versioning limits

Analyses retain a version-one snapshot, source extraction and annotation history. Corrections are not a full edit-and-reanalyze pipeline. Each disposition mutation creates an append-only REVIEW_EVENT containing the authenticated Cognito subject while REVIEW keeps the current state; corrections carry the same actor identifier. Historical matching identifies candidates using `invoice/provider/total`; it does not prove duplicate payment and does not yet offer a full prior-invoice comparison screen.

06 | Rules, amounts and uncertainty

Check	Inputs and interpretation
Bill reconciliation	Sum integer-paise line items plus explicit tax/GST, discount and signed round-off adjustments from a single invoice, then compare with its invoice total. Default tolerance: INR 1.00. Conflicting totals and ambiguous amounts prevent a pass.
Patient identity	Compare reliable identifiers across at least two distinct documents. Repeated fields in one document do not constitute a cross-document match.
Timeline	Compare admission, discharge and available procedure dates. Default allowance is one day outside the stay window. A same-day stay is not automatically suspicious.
Supporting evidence	Procedure/device charges prompt a request for relevant support. A report's presence alone does not prove it supports the charge; absence does not mean care did not occur.
Historical invoice	Compare a tenant-scoped fingerprint of invoice number, provider and total with prior records. A match is a review candidate only.

Status contract

PASS means the stated check succeeded on its available evidence. FINDING means a discrepancy needs human review. INSUFFICIENT_EVIDENCE means the check cannot be completed reliably. NOT_APPLICABLE means the defined condition does not apply. ERROR means the check or dependency could not run reliably. Never treat missing evidence or an AI outage as a pass.

Money and dates

Invoice total, amount paid, balance due and claimed amount are separate normalized fields; they are not interchangeable. Claimed amount remains unknown when absent. Arithmetic uses integer paise with configurable HALF_UP or HALF_EVEN rounding. Indian and Western thousands grouping are supported; malformed grouping and ambiguous decimal commas are rejected or flagged.

Numeric dates are ambiguous by default: DATE_ORDER=ISO_ONLY accepts ISO dates; DMY or MDY must be selected explicitly for numeric date formats. Stay-day allowance is a timeline assumption, not an implemented room-day or tariff billing policy. Repeated table headers and summary rows are excluded; explicit tax/GST, discount and signed round-off rows are normalized without inventing a missing sign. Values below the configurable `MinFieldConfidence` default of 80 require human confirmation and cannot create a clean result.

07 | Bounded AI: what it can and cannot do

Input and scope

Bedrock receives a bounded JSON evidence bundle: at most 120 fields and 40,000 encoded bytes. Report prose is limited to the first 40 nonempty Textract LINE blocks per discharge summary/supporting report, with excerpts capped at 240 characters. Billed procedure descriptions retain citations so they can be compared with report prose. Full raw extraction remains archived.

The Converse call uses the configured model ID, temperature zero and an output token bound. There are no agents, tool definitions, external browsing or model write privileges. Document text is explicitly untrusted data. The deployed IAM policy currently targets a regional foundation-model ARN; using an inference profile requires deliberate policy adjustments and verification.

Output contract

The response must have exactly one root key, findings. Each finding has checkId, title, summary, status, priority and evidencelds. Enum values, field types, lengths, finding counts and citation membership are validated. Unexpected keys, unsupported references, truncated/non-normal responses and invalid JSON do not produce trusted findings.

Failure behavior

If Bedrock is unavailable, disabled, exceeds bounds or returns invalid content, deterministic checks still run and an ERROR check is retained. This is visible as incomplete coverage or warnings, not fabricated semantic success. Failed/partial extraction prevents semantic comparison on an apparently complete packet.

Important limits

Citation validation proves that a referenced source exists; it does not mathematically prove that the model's sentence follows from that source. Prompt instructions and phrase checks are not a complete prompt-injection or clinical-safety guarantee. Human review remains essential. Bounded prose is not the full clinical record, and the app does not assess medical necessity, policy coverage, diagnostic correctness or visual forgery.

08 | Security, privacy and access boundaries

Identity and tenancy

The Cognito pool is administrator-provisioned. The immutable custom:tenant_id attribute cannot be assigned through the frontend's writable attributes. API Gateway validates issuer/audience; the Lambda additionally requires an ID token and a syntactically valid tenant claim. Every claim, analysis, finding, correction and document route resolves records within that tenant.

Storage and access

S3 public access is blocked, encryption at rest is enabled, HTTPS is required and source access uses short-lived signed URLs. Upload POSTs enforce content type, AES256 encryption and a 1-byte-to-15-MB range, and expire after 900 seconds by default; document read URLs expire after 300 seconds. DynamoDB encryption and point-in-time recovery are defined in SAM. These are template settings, not evidence of an active secure deployment.

Logs and model boundary

Application error logging records categories and identifiers, not source medical text or model bundles. The model never receives AWS credentials and has no tools to change application records. Avoid enabling payload/body logging when configuring API Gateway, Step Functions or debugging production lambdas.

What still needs a deployment/security review

- Test two real tenants against every HTTP route, not only the in-memory/emulated repository.
- Review generated IAM policies; SAM convenience policies are not a completed least-privilege audit.
- Validate the configured S3 current/noncurrent expiration, DynamoDB TTL delay, and deletion/incident procedures in the deployed account.
- Lambda log retention is explicit; add cloud alarms, budgets and operational ownership. Check the costs of Cognito advanced threat protection before deployment.
- Use synthetic medical data only. No HIPAA, GDPR or other compliance certification is claimed.

09 | Reliability, idempotency and limits

Duplicate requests and workers

Create/register/start operations use deterministic identifiers and tenant-scoped idempotency keys. DynamoDB acquires a conditional 180-second lease before running a request producer. Successful responses are cached; an active duplicate returns 409 and should retry with the same key. Failed producers release their lease. Reusing a key with a different packet/document/correction is rejected where implemented.

Textract jobs use a stable client request token and recorded job ID. A retried completed poll does not archive a second result. Completed analysis workers return the stored result without another model invocation. If a snapshot already exists after an interrupted write, it wins over recomputed findings. Transient Lambda invocation failures have bounded retries in Step Functions.

Failure handling

Partial extraction is labeled PARTIAL, not READY. Such fields are retained but excluded from automatic complete-packet checks. Workflow failures produce visible non-clean checks and FAILED status. A late failure handler must not overwrite an already completed analysis.

Bounded prototype

Boundary	Current behavior
Upload	10 documents maximum, 15 MB each, exactly one bill.
Per-document normalized data	More than 120 fields or 160 KB normalized JSON is rejected from inline review storage. Raw and normalized archives remain in S3.
Combined analysis fields	More than 220 KB fails the bounded analysis; use smaller packets.
Workflow	Standard workflow with three concurrent document branches, five-second polling and a one-hour execution timeout.
Model	120 evidence fields / 40 KB input bound; 1,800 output token cap.

Remaining reliability work

The maximum-shape local benchmark now exercises ten documents, 120 source-cited fields and the 220,000-byte combined-field bound. These measurements do not formally prove deployed DynamoDB, Lambda, Textract or Step Functions quotas. Top-level execution timeout reconciliation, transactional source-version pinning under racing starts, large-history pagination and long-running cloud load tests remain open. The prototype is not designed for unrestricted hospital-scale packets.

10 | Frontend structure and operation

Shared interface

theme.css holds the paper, charcoal and lime palette, text colors, focus rings and surfaces. hero.css provides Tailwind and the responsive split-image landing layout; styles.css applies the same low-noise visual system to the review workspace. The src/components/ui directory and @/ alias follow the shadcn-compatible structure. No second component root or new project initializer is required.

App.tsx owns queue/review/activity state, selected analysis, status polling, search, filters and actions. UploadDialog.tsx validates packet composition and preserves a retry key. SourceViewer.tsx handles PDF/image rendering, zoom, pagination, source geometry, correction forms and a small-screen drawer. api.ts separates real requests from explicit mock adapters; auth.ts handles Cognito sessions and token refresh.

Routes and configuration

Setting	Purpose
/	Landing hero and synthetic sample preview.
/review	Reviewer workspace. Query parameters preserve the selected analysis and queue/activity view.
/auth/callback	Cognito redirect callback. Host must rewrite this route to index.html.
VITE_APP_MODE	Local development defaults to mock; production builds default to production. Explicit mock builds use npm run build:demo.
VITE_API_BASE_URL	Deployed HTTP API endpoint.
VITE_COGNITO_*	Authority, client ID, redirect/logout URIs and hosted Cognito domain; see frontend/.env.example.

Implemented interaction safeguards

The logo returns home, navigation restores URLs, screen searches do not leak into other views, review tabs support arrow/Home/End keys, source zoom can shrink the page, and failed correction saves retain input without hiding evidence. Dialog labels, focus rings, reduced-motion styling and responsive layouts are implemented. Chromium automation now covers the landing page, queue, review, source viewer and upload dialog at 1536x960 and 390x844; six screenshots were inspected. Production Cognito authentication and a protected API read pass. Download behavior, additional browser engines, token refresh/logout, cross-tenant denial, and real signed-source rendering still need deployed acceptance.

11 | Verification and audit findings

Verification	Recorded result
Python dependency consistency	PASS
Python bytecode compilation	PASS
Python regression tests	PASS - 65 tests
Labeled rule quality benchmark	PASS
Maximum-shape packet benchmark	PASS
Eight-page OCR artifact validation	PASS
CloudFormation / SAM lint	PASS
AWS verifier shell syntax	PASS
React interaction tests	PASS - 14 tests
TypeScript + Vite build	PASS
Frontend formatting	PASS
Desktop/mobile browser workflow	PASS

Local browser workflow: PASS. Manual screenshot review: PASS. Amplify hosting, Cognito login and a protected API read are live-verified; Textract, Bedrock and signed-source behavior remain unverified.

What the tests actually establish

Python tests exercise safe arithmetic, explicit adjustments, low-confidence gating, repeated headers, multilingual labels, citation validity, tenant isolation, duplicate requests, partial failures, snapshot retries and five synthetic packet categories. Moto tests emulate DynamoDB/S3 with real SDK serialization. Worker tests stub Textract/model dependencies to inspect job reuse, pagination, pinned source versions and failure behavior.

React tests run in jsdom. They cover queue selection, filtering, reviewer dispositions, corrections, failed-save retry, mock upload submission, tab keyboard behavior, evidence navigation, report-download initiation, hero sample dialogs and animation controls. A separate Playwright flow exercises desktop/mobile Chromium navigation, reviewer mutation, audit activity, evidence, and dialogs while rejecting page/console errors, duplicate IDs, missing image alt text, and viewport overflow. It does not establish real PDF download behavior or cloud service correctness.

Improvements from this readiness pass

- Required explicit numeric date order; rejected malformed/sign-conflicted/unsafe amounts and missing/nonfinite geometry.
- Preserved legitimate procedure rows, source-cited bounded prose and normalized S3 archives; made partial extraction and size limits explicit.
- Hardened polling/analysis retries, snapshot recovery and completed-result protection; stabilized correction identifiers.
- Added S3/DynamoDB/Lambda-log retention, authenticated append-only review events, browser acceptance, a full local verifier and CI gates.

Scope of confidence

The supplied CI has not run on GitHub. SAM lint does not prove deployment or IAM access. Current npm and Python advisory scans report no known vulnerabilities; this is not a penetration test and advisory data changes. Live cloud acceptance remains outstanding.

12 | AWS deployment and live acceptance gate

Before deploying

1. Install AWS CLI and SAM CLI, authenticate using an approved account/role, and select a region. Do not paste credentials into source code or this PDF.
2. Verify a Converse-compatible Bedrock foundation model in that region. A model list alone does not establish invocation permission. Keep the chosen identifier configurable.
3. Choose a static frontend host, enable HTTPS and SPA rewrites, and decide exact origin/callback/logout URLs. The SAM template can create S3/CloudFront hosting or accept an external HTTPS host such as Amplify.
4. Set a spend budget and inspect IAM, Cognito tier, retention and service quotas. Use synthetic files only.

Local and deployment commands

```
.venv/bin/python scripts/verify_project.py
AWS_REGION=<region> BEDROCK_MODEL_ID=<model-id> ./scripts/verify_aws.sh
sam build
sam validate --lint
sam deploy --guided --region <region>
```

SAM parameters include BedrockModelId, EnableCloudFront, AllowedOrigin, CognitoCallbackUrl, CognitoLogoutUrl, MoneyTolerancePaise, StayDayAllowance, RoundingMode, DateOrder, RetentionDays and LogRetentionDays. Leave the origin, callback and logout parameters blank to use generated CloudFront hosting. For Amplify or another HTTPS host, set EnableCloudFront=false and provide all three URLs. Use DateOrder=ISO_ONLY unless numeric date convention is explicitly known. Configure frontend environment variables from stack outputs and rebuild; Vite variables are build-time values.

Mandatory live checks before calling it AWS-demo ready

- Provision two Cognito reviewers in distinct tenants; verify login, refresh, logout and unauthorized record denial.
- Upload a small synthetic bill, summary and report. Confirm S3 version pinning and HTTP 202 analysis ID, then inspect processing transitions to completion/warnings.
- Confirm real Textract extraction and every evidence page/block/geometry against the original PDF. Test a partial/failed document.
- Confirm a real Bedrock invocation and fail-closed handling of invalid or unsupported citations. Review generated content manually.
- Retry requests, inspect latest report/dispositions/corrections, test browser Back/Forward and review on narrow and wide screens.

The verifier probes Textract GetDocumentAnalysis reachability and sends a minimal synthetic Bedrock request, which can incur charges. It does not prove StartDocumentAnalysis or all S3/DynamoDB/Cognito/Step Functions permissions.

13 | Hackathon demo and submission plan

Five-minute demonstration

Time	Story and action
0:00-0:30	Explain the reviewer problem: hospital claim packets contain amounts and facts spread across documents. ClaimLens organizes evidence, not fraud verdicts.
0:30-1:45	Open the inconsistent sample. Show INR 248,500 stated total versus INR 241,500 line items, a difference of INR 7,000. Follow the bill citation and source details.
1:45-2:45	Show a timeline discrepancy and missing support request. Explain why missing evidence is not proof a procedure did not occur.
2:45-3:30	Acknowledge/resolve a finding, add a correction annotation, inspect activity and export the JSON report. State that corrections do not yet rerun rules.
3:30-4:15	Switch to the ambiguous and same-day samples. Show that uncertainty and legitimate edge cases do not become fraud accusations.
4:15-5:00	Explain the AWS architecture and safeguards. State the actual deployment status. If cloud setup is incomplete, label this an offline synthetic demo.

Rehearsal and backup

Run the local verification script, confirm its browser check, test the report button and rehearse on the presentation screen. Keep a local explicit-mock build available. Do not hide a cloud outage by pretending mock results came from AWS. Keep the inspected screenshots and a short recorded demo available as backup.

Submission checklist

Confirm event-specific AWS-service requirements, eligible region/account, team size, repository/license requirements, mandatory video length, judging criteria and submission deadline. Prepare a concise problem statement, working link, repository, architecture, synthetic-data statement, test evidence, limitations and cost notes. These rules cannot be confirmed until the hackathon is identified.

Decision

Use this project as a credible deployed prototype with an accurate status description: AWS hosting, authentication and protected API access are live, while Textract/Bedrock analysis remains activation-blocked. Do not describe it as a completed live document-processing system until the acceptance gate on the previous page passes. Real medical use is out of scope.

14 | Reference map and next priorities

Code and artifacts to inspect

Artifact	Why it matters
template.yaml	Actual AWS resources, parameters, permissions and workflow definition.
backend/claimlens/ + frontend/src/	Backend extraction/check/model logic and the reviewer interface, theme, API and auth wiring.
synthetic/ + benchmarks/	Local rule packets plus ground truth and the eight-page live OCR corpus.
scripts/evaluate_quality.py	Local status accuracy, finding precision/recall/F1, confusion and false-clean gate.
scripts/evaluate_live_aws.py + scripts/run_live_acceptance.py	Paid live Textract/Bedrock metrics plus sanitized Cognito, tenant, S3, workflow, report and source-overlay acceptance.
tests + docs/verification-results.json	Executable regression evidence and the timestamped local verification report.
scripts/verify_project.py	Compilation, tests, benchmarks, build, formatting, SAM lint and desktop/mobile browser acceptance.
scripts/verify_aws.sh	Initial cloud capability probe; requires credentials, selected region/model and AWS CLI.

Priorities after the hackathon gate

First run the included live OCR/model benchmark in the selected AWS region and verify production source overlays. Next implement correction-driven reanalysis. Then expand the labeled layout corpus, deployed load testing, prior-invoice comparison, retention observation and timeout recovery. Do not add agents, vector databases, model training or forgery detection until these fundamentals are reliable.

Official references checked during this assessment

1. Amazon Bedrock, Model availability and compatibility:
<https://docs.aws.amazon.com/bedrock/latest/userguide/models.html>
2. AWS Step Functions, Service quotas: <https://docs.aws.amazon.com/step-functions/latest/dg/service-quotas.html>
3. Amazon Cognito, User pool feature plans:
<https://docs.aws.amazon.com/cognito/latest/developerguide/cognito-sign-in-feature-plans.html>

AWS details change by region and account. Verify these official references again when choosing the deployment settings. Product behavior described in this guide is based on the inspected source code and local test results, not on an external review or a deployed security certification.